

GSOC 2026 PROJECT PROPOSAL

A DeepChem-compatible End-to-End RFDiffusion Implementation

Organization: DeepChem

Author:
Shubham Bari

TABLE OF CONTENTS

1. INTRODUCTION.....	3
2. ABSTRACT.....	3
3. TECHNICAL DETAILS.....	4
3.1 BACKGROUND.....	4
A. The Three-Track Backbone (RF2).....	4
B. The Diffusion Engine (Denoising Logic).....	5
C. Leveraging Equivariance Primitives.....	5
4. DEEPCHEM INTEGRATION.....	6
5. BENCHMARKING AND VALIDATION.....	9
5.1 The In Silico Self-Consistency Pipeline.....	9
5.2 Designability and Confidence Metrics.....	10
5.3 Structural Diversity.....	11
5.4 Task Specific Benchmarks.....	11
6. PROJECT OBJECTIVES.....	11
7. PROJECT TIMELINE.....	13
8. OTHER INFORMATION.....	15
8.1 ACADEMIC BACKGROUND.....	15
8.2 PAST EXPERIENCES.....	15
8.3 COMMITMENTS.....	16
9. REFERENCES.....	16

1. INTRODUCTION

- **Name:** Shubham Bari
- **Email:** shubham.bari001@gmail.com, shubhamvbari@gmail.com
- **Contact Number:** +91 9029769086
- **University:** Thakur College of Engineering and Technology (Affiliated with University of Mumbai)
- **Majors:** B.Tech in AI and Data Science
- **GitHub:** github.com/shubham-bari
- **LinkedIn:** [Shubham-Bari](#)
- **CGPA:** 9.19/10

2. ABSTRACT

This project aims to implement state-of-the-art protein design models, RFDiffusion and RFDiffusion-2, within the DeepChem framework using a clean, end-to-end PyTorch pipeline. The goal is to make advanced generative protein modeling accessible through standardized DeepChem abstractions such as TorchModel, Dataset, and Featurizer, facilitating integration with existing deepchem workflows. The model leverages diffusion-based generative techniques to transform random noise into realistic protein structures, built on a RoseTTAFold inspired architecture that jointly captures sequence, pairwise interactions, and 3D geometry. By incorporating SE(3)-equivariant operations, the system ensures physically consistent predictions while maintaining modularity through the use of existing equivariant libraries.

This design lowers the barrier for researchers to train, evaluate, and generate proteins using cutting-edge methods, while maintaining extensibility for future research and models. To validate performance, the project includes a rigorous benchmarking pipeline unit tests such

as self-consistency checks, structural similarity metrics, and diversity analysis across tasks like monomer generation, motif scaffolding, and binder design.

3. TECHNICAL DETAILS

3.1 BACKGROUND

RoseTTAFold and Diffusion

— RoseTTAFold is a deep-learning model that jointly processes 1D sequences, 2D residue-pair features, and 3D coordinates to predict protein structures with high accuracy. It enables long-range communication across these representations, making it a strong foundation for generative protein design.

— RFDiffusion builds on Denoising Diffusion Probabilistic Models (DDPMs), where a forward process gradually adds noise to protein coordinates until they become random, and a reverse process learns to recover the original structure.

Deepchem

— “deepchem” is a deep-learning library for chemistry and biology that provides unified abstractions for datasets, featurizers, and models, including TorchModel for PyTorch integration.

— By implementing RFDiffusion-2 as a TorchModel on top of DeepChem Dataset instances, the model can plug into existing workflows for training, evaluation, and benchmarking.

3.2 PROPOSED ARCHITECTURE

A. The Three-Track Backbone (RF2)

The core of this model consists of a specialized transformer architecture that updates three distinct representations of the protein in parallel:

- **The 1D Track:** Processes residue-level information, including the amino acid identity and the relative position in the chain.

- **The 2D Track:** A pairwise representation that captures the spatial relationships between every residue in the protein.
- **The 3D Track:** This is the geometric track where the physical coordinates and 3×3 rotation matrices are updated. It uses an SE(3)-Equivariant Transformer to ensure the model respects 3D spatial symmetries.

B. The Diffusion Engine (Denoising Logic)

The model will implement protein generation by learning to remove gaussian noise from a sample until it reaches a stable state. The architecture will implement two distinct diffusion processes occurring simultaneously:

- **Translation Diffusion:** Occurs in Euclidean space (R^3) for the $C\alpha$ coordinates of each residue.
- **Rotation Diffusion:** Occurs on the $SO(3)$ manifold for the orientation of each residue.
- **Self-Conditioning:** To stabilize generation, the architecture will include a memory mechanism where the model's prediction of the final structure at step $t+1$ is fed back into the model as an input for step t .

C. Leveraging Equivariance Primitives

- Implements equivariant operations inspired by the **e3nn** framework directly **within the DeepChem** codebase to avoid introducing external dependencies and ensure long-term stability.
- Core components such as spherical harmonics, tensor products, and irreducible representation (**Irreps**) handling will be **reimplemented**.
- These primitives will be used to capture 3D protein geometry while preserving rotational equivariance, with features represented as Irreps to guarantee consistent behavior under spatial transformations.

4. DEEPCHEM INTEGRATION

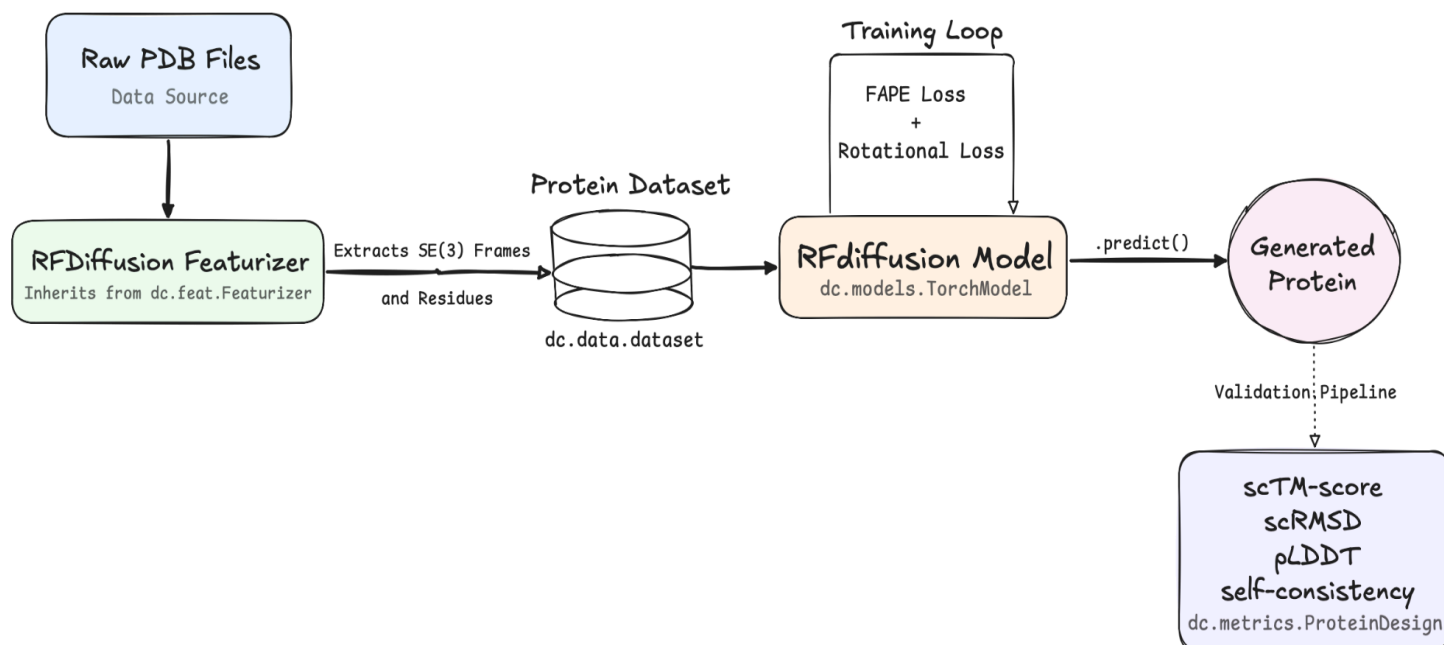


Fig. 1 Workflow of RFDiffusion model integrated with deepchem

A. ProteinDataset and Data Loading

```
class ProteinDataset(dc.data.Dataset):
    """
    A dataset class tailored for protein diffusion rather than simple SMILES or fingerprints

    This dataset will store backbone coordinates, residue types, and local frames (rotation + translation)
    extracted from PDB files. It will also support efficient streaming and batching
    """

    def __init__(self, data_source, featurizer):
        # Initialization logic
        pass

    def iterbatches(self, batch_size):
        """
        Yields batches of SE(3) frames and corresponding protein features.
        """
        pass
```

— We will implement a `ProteinDataset` that inherits from '`dc.data.Dataset`', tailored for protein diffusion rather than simple SMILES or fingerprints. This dataset will store backbone coordinates, residue types, and local frames (rotation +

translation) extracted from PDB files and will support efficient streaming and batching.

B. Specialized Protein Featurization

— To transform raw PDB data into diffusion-ready inputs, we will extend the existing **EquivariantFeaturizer** within DeepChem to support residue-level representations.

```
#Featurizer for RFDiffusion
class RFDiffusionFeaturizer(dc.featurizer.EquivariantFeaturizer):
    """
    Transforms raw PDB data into diffusion-ready inputs by extending
    the existing Equivariant Featurizer within DeepChem.

    This extended featurizer will parse structural files, compute residue-level features,
    and generate SE(3) frames for each residue, enabling compatibility with
    diffusion-based protein modeling while maintaining equivariant properties.
    """

    def __init__(self, **kwargs):
        super().__init__(**kwargs)

    def featurize(self, raw_pdb_files):
        """
        Extracts SE(3) frames and residues from raw PDB data sources.
        """
        # Logic to parse PDB, compute 1D, 2D, and 3D features while preserving equivariance
        pass
```

— This extended featurizer will parse structural files, compute residue-level features, and generate SE(3) frames for each residue, enabling compatibility with diffusion-based protein modeling while maintaining equivariant properties.

C. TorchModel Wrapper and API

— The core RFDiffusion architecture will be implemented as a standalone PyTorch nn.Module and this module will then be wrapped within a deepchem-compatible RFDiffusionModel class inheriting from '**dc.models.TorchModel**'.

```

#Model Wrapper for RFDiffusion
class RFDiffusionModel(dc.models.TorchModel):
    """
    DeepChem-compatible wrapper for the RFDiffusion architecture[cite: 91].

    This class encapsulates a PyTorch nn.Module implementing
    the core RFDiffusion logic (RF2 backbone diffusion process)
    and exposes it through DeepChem's standardized TorchModel interface for training, evaluation,
    and generative inference[cite: 92]. By inheriting from TorchModel,
    it automatically gains support for model saving/loading,
    logging via TensorBoard and multi-GPU training.
    """

    def __init__(self, model: nn.Module, loss_fn, **kwargs):
        super().__init__(model=model, loss=loss_fn, **kwargs)

    def predict(self, inputs):
        """
        Runs the reverse diffusion process to generate protein structures
        from input constraints (e.g., length, motifs, or partial structures).
        """
        return self.model.sample(inputs)

```

- By building on TorchModel, the implementation automatically gains support for model saving/loading, logging via TensorBoard and multi-GPU training.

D. Custom Loss and Generative Inference

- We will define a custom loss function that subclasses DeepChem's 'Lossclass' and handles targets such as Frame Aligned Point Error (**FAPE**) and SO(3) manifold distances between predicted and target rotations.
- The iterative reverse-diffusion process will be wrapped into the model's .predict() method, so users can generate new protein structures by specifying simple constraints.

E. Protein Design Metrics Module

- We will implement a '**dc.metrics.ProteinDesign**' module containing specialized metrics to evaluate success in protein generation tasks.

— We will also track **scTM-score** and Prediction Confidence (**pLDDT**), which reports the mean per-residue confidence from a refolding step.

5. BENCHMARKING AND VALIDATION

To validate the DeepChem implementation of RFDiffusion, We will establish a rigorous evaluation pipeline that mirrors the industry standard **refolding** protocol. This ensures the generated backbones are physically plausible and foldable in a biological context.

5.1 The In Silico Self-Consistency Pipeline

The primary method for validating generative protein models is a three step check where for every backbone generated by the RFDiffusionModel, we will perform:

- **Sequence Design:** Using **ProteinMPNN** to generate different sequences that fit the new backbone.
- **Structure Prediction:** Using **ESMFold** or **AlphaFold2** to refold these sequences from scratch.
- **Comparison:** Calculating the alignment between the original generated backbone and the predicted structure to verify that the model is learning the underlying physics of protein folding rather than just memorizing noise.

```

class SelfConsistencyPipeline:
    """
    Implements a three step validation check to ensure generated protein backbones
    are physically plausible and foldable in a biological context.
    """

    def __init__(self, sequence_generator, structure_predictor):
        self.sequence_generator = sequence_generator #ProteinMPNN
        self.structure_predictor = structure_predictor #SMFold or AlphaFold2

    def validate_backbone(self, generated_backbone):
        """
        Executes the self-consistency check for a single generated backbone.

        Step 1: Sequence Design - Generate sequences using ProteinMPNN.
        Step 2: Structure Prediction - Refold sequences using ESMFold or AlphaFold2.
        Step 3: Comparison - Calculate alignment (scRMSD/scTM-score).
        """
        # Step 1: Design sequences that fit the generated backbone geometry
        designed_sequences = self.sequence_generator.design(generated_backbone)

        results = []
        for sequence in designed_sequences:
            # Step 2: Refold the designed sequence from scratch
            predicted_structure = self.structure_predictor.predict(sequence)

            # Step 3: Calculate alignment between original and predicted structures
            # This verifies the model learned underlying physics rather than memorizing noise
            alignment_score = self.calculate_alignment(
                generated_backbone,
                predicted_structure
            )
            results.append(alignment_score)

        return results

    def calculate_alignment(self, original, predicted):
        """
        Calculates metrics like scRMSD and scTM-score to quantify success.
        """
        pass

```

5.2 Designability and Confidence Metrics

We will implement a suite of metrics within a **'dc.metrics.ProteinDesign'** module to quantify model success.

- **Self-Consistency RMSD** (scRMSD): Measures the average distance between the C α atoms of the design and the refolded prediction.

- **scTM-Score:** A length independent metric where a score > 0.5 indicates the same global fold.
- **Prediction Confidence (pLDDT):** We will track the mean pLDDT from the refolding step.

5.3 Structural Diversity

— We will perform hierarchical clustering on sets of generated proteins using a calculated TM-score threshold.

— High diversity or a high '**clusters/samples**' ratio indicates that the model is exploring the full range of possible architectures rather than suffering from mode collapse.

5.4 Task Specific Benchmarks

- **Monomer Generation:** Unconditional design of novel proteins up to 600 residues.
- **MotifBench:** Evaluation on the MotifBench suite of 30 problems, focusing on the model's ability to scaffold functional motifs.
- **Binder Design:** Measuring the success rate in creating new binders for therapeutic targets, evaluated via predicted Aligned Error (**pAE**) at the interface.
- **Enzyme Design (AME Benchmark):** We will also use the Atomic Motif Enzyme (**AME**) benchmark to verify the model's precision in placing catalytic sidechains for specific chemical reactions.

6. PROJECT OBJECTIVES

The objective of this project is to implement a robust, end-to-end generative framework for protein design within the DeepChem library, focusing on the RFDiffusion-2 architecture. This includes:

1) Literature Review and Architecture Mapping

- a) Conducting a comprehensive review of the RoseTTAFold 2 (RF2) multi-track architecture and its integration into the

Denoising Diffusion Probabilistic Model (DDPM) framework.

- b) Identifying the existing equivariance primitives within DeepChem and external libraries like e3nn to be leveraged for the geometric transformer.

2) DeepChem Infrastructure Integration

- a) Developing a specialized ProteinDataset and RFDiffusionFeaturizer to handle the parsing and streaming of structural PDB data.
- b) Implementing a native 'dc.models.RFDiffusionModel' class that inherits from dc.models.TorchModel to provide a standardized API for the community.
- c) Creating clean documentation for easy implementation and debugging by the users.

3) Experimenting with Generative Design Tasks

- a) Training will be conducted on curated structural datasets derived from the **Protein Data Bank**, with redundancy reduction and quality filtering applied using resources such as the **PISCES dataset** and **RCSB PDB filtered sets** to ensure high-quality, diverse training data.
- b) Integrating self-conditioning mechanisms to improve the stability and resolution of the generated structural frames.

4) Rigorous Benchmarking and Evaluation

- a) Establishing an automated validation pipeline using **ProteinMPNN** for sequence design and **ESMFold** for structural refolding.
- b) Quantifying model performance using industry-standard metrics such as **scRMSD**, **scTM-score**, and **pLDDT**.

7. PROJECT TIMELINE

Week Number	Work Description
Community Bonding Period	During this Period, I would like to engage and interact with my mentors and deepchem community, and examine past work done by contributors to see their methodology and workflow while also familiarising myself with key biology and chemistry concepts from related papers on protein-folding models.
Week 1	Conduct a literary review on diffusion models, protein folding models and their working mechanisms. Carefully review previous implementations of similar models and gain a deep understanding of the structures and techniques used.
Week 2	I will implement the ProteinDataset, which parses the PDB files to extract residue, frames and coordinates.
Week 3	I will implement the RFDiffusionFeaturizer to generate SE(3) frames, residue-level features, and integrate it with DeepChem's featurizer API.
Week 4	I will implement the core 3-track backbone architecture (1D, 2D, 3D tracks) using equivariant primitives and validate forward passes.

Week 5	I will implement the diffusion engine, including translation diffusion in R^3 and rotation diffusion on $SO(3)$, along with noise schedules.
Week 6	I will incorporate self-conditioning into the model, improve training stability, and evaluate initial convergence. A mid-phase report will be prepared.
Week 7	I will wrap the model into a DeepChem-compatible TorchModel, enabling training, logging, and checkpointing within the framework.
Week 8	I will implement custom loss functions such as FAPE and rotation-aware losses, and begin training on smaller datasets to validate learning.
Week 9	I will build the evaluation pipeline, including self-consistency checks using ProteinMPNN and structure prediction tools like ESMFold.
Week 10	I will conduct benchmarking across tasks such as monomer generation and motif scaffolding, and evaluate metrics like scRMSD, scTM-score, and pLDDT.
Week 11	I will optimize the model for performance by improving inference speed and memory usage

	using mixed precision, efficient batching, and other techniques.
Week 12	I will finalize all deliverables, including clean code, complete documentation, final report, and preparation for submission and evaluation.

8. OTHER INFORMATION

8.1 ACADEMIC BACKGROUND

I am currently pursuing a B.Tech in Artificial Intelligence and Data Science from Thakur College of Engineering and Technology, Mumbai, affiliated with the University of Mumbai, with a CGPA of 9.19. My coursework includes advanced topics such as machine learning theory, deep learning, reinforcement learning, and modern AI algorithms.

Beyond formal coursework, I have independently explored areas not covered in the curriculum, focusing on research-driven topics and building projects across multiple domains in machine learning and artificial intelligence.

8.2 PAST EXPERIENCES

I have worked as a research intern with UNSW, Sydney, on gradient inversion attacks across various machine learning systems, including large language models, neural networks, and diffusion models. In this work, I explored the reconstruction of training data from shared gradients in federated learning settings, gaining hands-on experience in privacy vulnerabilities of modern ML systems. I am also collaborating with Dr. Vincenzo Sorrentino at NUS on applying machine learning techniques, including diffusion models and ensemble methods, to virtual screening of proteins for PROTAC docking and drug discovery applications.

Additionally, I actively contribute to research-oriented projects within the Cohere Labs community, focusing on principles of diffusion learning and reinforcement learning. These experiences have collectively strengthened my ability to work on complex, interdisciplinary ML research problems and prepared me for the implementation of this project.

8.3 COMMITMENTS

My degree program is quite flexible, allowing me to adjust my schedule during the 7th semester and focus on any internships or work I may have. Previously, I balanced my Research Intern with coursework later in the day, and during GSoC, if selected, my focus will shift from studies to the project while maintaining my professional responsibilities. In case of unavoidable circumstances leading to a temporary reduction in working hours, I will ensure to compensate for the time in the following weeks. I will communicate any such situations with my mentors well in advance to ensure smooth project progress. For regular updates and discussions, I will be available via Teams, Zoom, and Google Meet in the Indian Standard Time Zone (GMT +5:30).

9. REFERENCES

- 1) Ho, Jonathan, Ajay Jain, and Pieter Abbeel. "Denoising diffusion probabilistic models." *Advances in neural information processing systems* 33 (2020): 6840-6851.
- 2) Rohith Krishna *et al.*, Generalized biomolecular modeling and design with RoseTTAFold All-Atom. *Science* 384, ead12528 (2024). DOI:10.1126/science.adl2528
- 3) Geiger, Mario, and Tess Smidt. "e3nn: Euclidean neural networks." *arXiv preprint arXiv:2207.09453* (2022).
- 4) Fuchs, Fabian, et al. "Se (3)-transformers: 3d roto-translation equivariant attention networks." *Advances in neural information processing systems* 33 (2020): 1970-1981.

- 5) Watson, J.L., Juergens, D., Bennett, N.R. *et al.* De novo design of protein structure and function with RFdiffusion. *Nature* 620, 1089–1100 (2023). <https://doi.org/10.1038/s41586-023-06415-8>
- 6) Ahern, W., Yim, J., Tischer, D. *et al.* Atom-level enzyme active site scaffolding using RFdiffusion2. *Nat Methods* 23, 96–105 (2026). <https://doi.org/10.1038/s41592-025-02975-x>
- 7) Jumper, J., Evans, R., Pritzel, A. *et al.* Highly accurate protein structure prediction with AlphaFold. *Nature* 596, 583–589 (2021). <https://doi.org/10.1038/s41586-021-03819-2>
- 8) Wu, K.E., Yang, K.K., van den Berg, R. *et al.* Protein structure generation via folding diffusion. *Nat Commun* 15, 1059 (2024). <https://doi.org/10.1038/s41467-024-45051-2>
- 9) J. Dauparas *et al.*, Robust deep learning-based protein sequence design using ProteinMPNN. *Science* 378, 49–56 (2022). DOI:10.1126/science.add2187
- 10) Zeming Lin *et al.*, Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science* 379, 1123–1130 (2023). DOI:10.1126/science.ade2574
- 11) Aier, I., Varadwaj, P. & Raj, U. Structural insights into conformational stability of both wild-type and mutant EZH2 receptor. *Sci Rep* 6, 34984 (2016). <https://doi.org/10.1038/srep34984>
- 12) Guo HB, Perminov A, Bekele S, Kedziora G, Farajollahi S, Varaljay V, Hinkle K, Molinero V, Meister K, Hung C, Dennis P, Kelley-Loughnane N, Berry R. AlphaFold2 models indicate that protein sequence determines both structure and dynamics. *Sci Rep.* 2022 Jun 23;12(1):10696. doi: 10.1038/s41598-022-14382-9. PMID: 35739160; PMCID: PMC9226352.

13) Protein language model supervised motif-scaffolding design with GPDL, International Journal of Biological Macromolecules, Volume 331, Part 2, 2025, 148441, ISSN 0141-8130, <https://doi.org/10.1016/j.ijbiomac.2025.148441>.